

inter-process communication (IPC)

Processes can coordinate and interact with one another using a method called **inter-process communication (IPC)**. Through facilitating process collaboration, it significantly contributes to improving the effectiveness, modularity, and ease of software systems.

Types of Process

- **Independent process**
- **Co-operating process**

An independent process is not affected by the execution of other processes while a co-operating process can be affected by other executing processes. There are many situations when cooperative nature can be utilized for increasing computational speed, convenience, and modularity. Inter-process communication (IPC) is a mechanism that allows processes to communicate with each other and synchronize their actions. The communication between these processes can be seen as a method of cooperation between them. Processes can communicate with each other through both:

Methods of IPC

- **Shared Memory**
- **Message Passing**

An operating system can implement both methods of communication.

Communication between processes using shared memory requires processes to share some variable, and it completely depends on how the programmer will implement it. One way of communication using shared memory can be imagined like this: Suppose process1 and process2 are executing simultaneously, and they share some resources or use some information from another process. Process1 generates information about certain computations or resources being used and keeps it as a record in shared memory. When process2 needs to use the shared information, it will check in the record stored in shared memory and take note of the information generated by process1 and act accordingly. Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes.

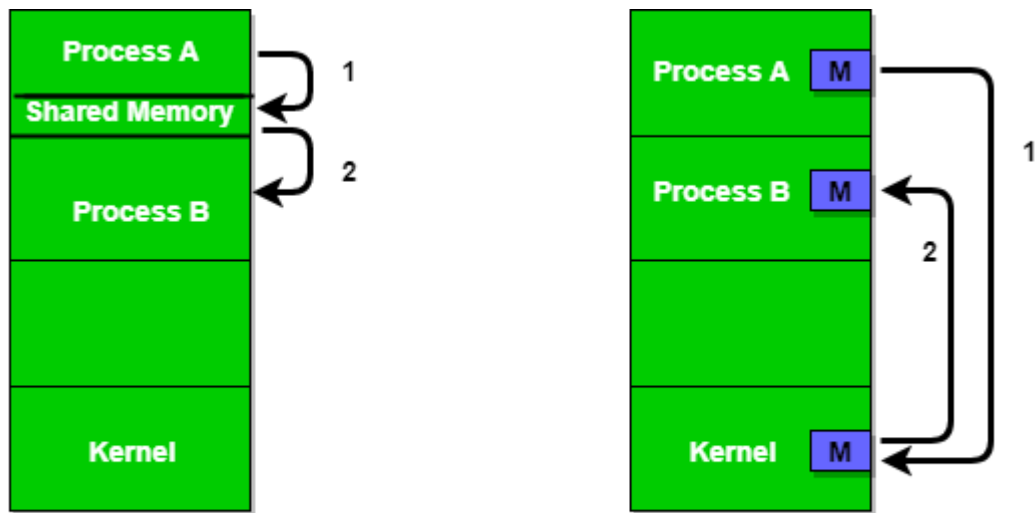


Figure 1 - Shared Memory and Message Passing

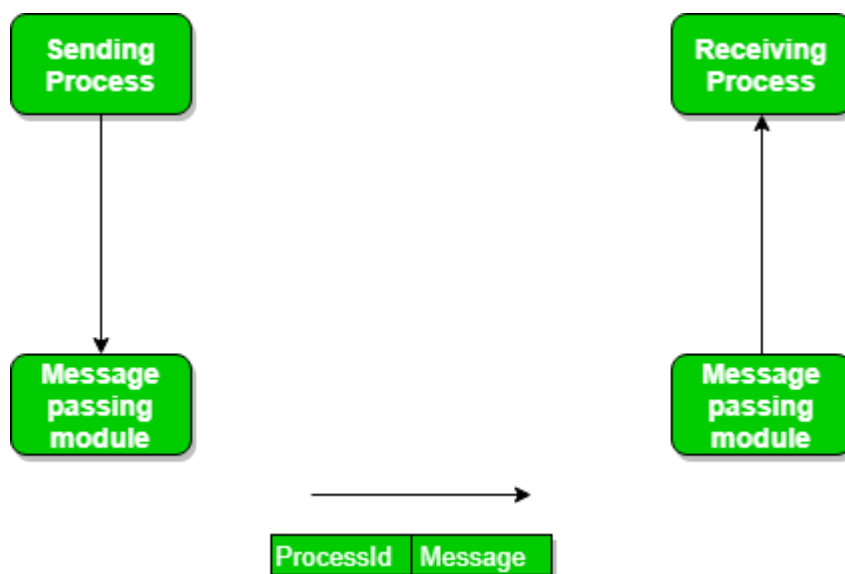
Message Passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.

For example – chat programs on World Wide Web.

Now let us discuss the message passing step by step.

Step 1 – Message passing provides two operations which are as follows –

- Send message
- Receive message



Messages sent by a process can be either fixed or variable size.

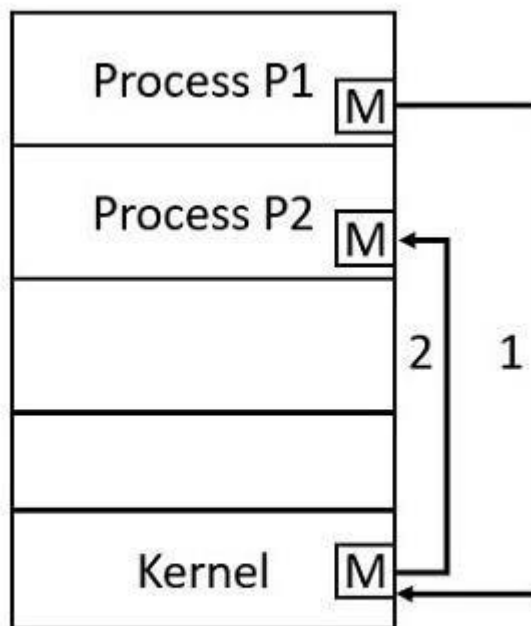
Step 2 – For fixed size messages the system level implementation is straight forward. It makes the task of programming more difficult.

Step 3 – The variable sized messages require a more system level implementation but the programming task becomes simpler.

Step 4 – If process P1 and P2 want to communicate they need to send a message to and receive a message from each other that means here a communication link exists between them.

Step 5 – Methods for logically implementing a link and the send() and receive() operations.

Given below is the structure of message passing technique –



Message Passing System

Characteristics

The characteristics of Message passing model are as follows –

- Mainly the message passing is used for communication.
- It is used in distributed environments where the communicating processes are present on remote machines which are connected with the help of a network.
- Here no code is required because the message passing facility provides a mechanism for communication and synchronization of actions that are performed by the communicating processes.
- Message passing is a time consuming process because it is implemented through kernel (system calls).
- It is useful for sharing small amounts of data so that conflicts need not occur.
- In message passing the communication is slower when compared to shared memory technique.

Race condition

A **race condition** is a situation that occurs when multiple threads or processes attempt to access and modify shared resources concurrently, leading to unpredictable and erroneous outcomes. This typically happens in a multithreaded or multiprocessing environment where the execution order of threads or processes affects the final result¹.

Key Characteristics of Race Conditions:

1. **Concurrency:** Race conditions occur in environments where multiple threads or processes execute concurrently.
2. **Shared Resources:** The threads or processes must share common resources, such as variables, data structures, files, or memory.
3. **Timing Dependence:** The final outcome depends on the timing or order of execution of the threads or processes.

Example of Race Conditions

Tube Light with Multiple Switches

Consider a tube light controlled by two switches. If both switches are turned on simultaneously, the light may end up in an inconsistent state, such as being off despite both switches being on. This is analogous to race conditions in computing, where simultaneous read and write operations can lead to data corruption¹.

Producer-Consumer Problem

In the **producer-consumer problem**, the producer generates data and the consumer reads it. If the consumer reads data faster than the producer can produce it, or vice versa, it can lead to inconsistencies. Proper synchronization is required to ensure that the producer and consumer operate in harmony¹.

Prevention of Race Conditions

Using Locks

Locks, such as **mutexes** and **semaphores**, can be used to ensure that only one thread or process accesses a critical section at a time. This prevents conflicting operations and ensures data consistency².

Atomic Operations

Treating critical sections as atomic operations ensures that they are executed as a single, indivisible step, preventing other threads from interfering during execution³.

Volatile and Synchronized Keywords in Java

In Java, the **volatile** keyword ensures that a variable's value is always read from the main memory, preventing threads from caching it locally. The **synchronized** keyword can be used to lock methods or code blocks, ensuring that only one thread can execute them at a time¹.

Conclusion

Race conditions are a significant challenge in concurrent programming, leading to unpredictable and erroneous outcomes. By using synchronization techniques such as locks,

semaphores, and atomic operations, race conditions can be effectively prevented, ensuring data consistency and system reliability.

#Critical Section

A **critical section** is a part of the code where shared resources are accessed. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device. Only one thread or process should execute in the critical section at a time to prevent data corruption and ensure consistency².

The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.

The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

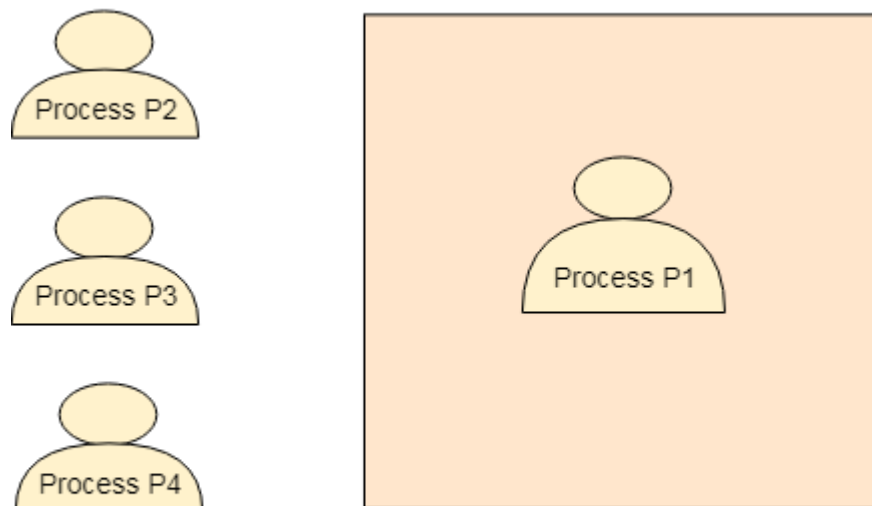
In order to synchronize the cooperative processes, our main task is to solve the critical section problem. The critical section problem can be solved by ensuring **mutual exclusion**, **progress**, and **bounded waiting**³.

Primary

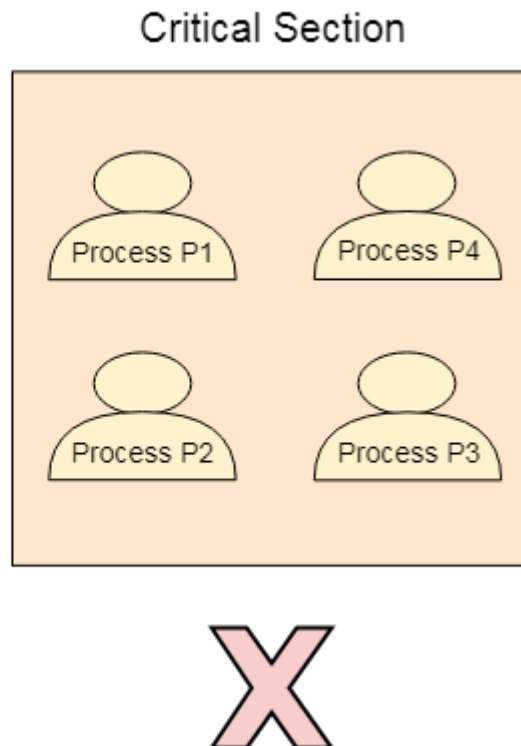
1. **Mutual Exclusion**

Our solution must provide mutual exclusion. By Mutual Exclusion, we mean that if one process is executing inside critical section then the other process must not enter in the

Critical Section



critical section



2. Progress

Progress means that if one process doesn't need to execute into critical section then it should not stop other processes to get into the critical section.

Secondary

1. Bounded Waiting

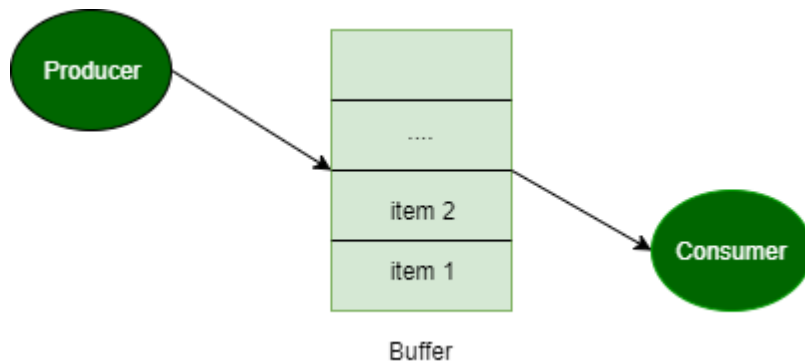
We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

2. Architectural Neutrality

Our mechanism must be architectural natural. It means that if our solution is working fine on one architecture then it should also run on the other ones as well.

Peterson's Algorithm in Process Synchronization

The producer-consumer problem (or bounded buffer problem) describes two processes, the producer and the consumer, which share a common, fixed-size buffer used as a queue. Producers produce an item and put it into the buffer. If the buffer is already full then the producer will have to wait for an empty block in the buffer. Consumers consume an item from the buffer. If the buffer is already empty then the consumer will have to wait for an item in the buffer. Implement Peterson's Algorithm for the two processes using shared memory such that there is mutual exclusion between them. The solution should have free from [synchronization](#) problems.



Key Properties of Peterson's Algorithm

Peterson's Algorithm ensures mutual exclusion, progress, and bounded waiting:

- **Mutual Exclusion:** No two processes can be in the critical section simultaneously.
- **Progress:** If no process is in the critical section, and there are processes that wish to enter the critical section, one of the waiting processes will be allowed to enter the critical section in a finite amount of time.
- **Bounded Waiting:** There exists a limit on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Advantages and Disadvantages

Advantages:

- Simple and easy to understand.
- Guarantees mutual exclusion and can be implemented in user mode.
- Does not require special hardware instructions.

Disadvantages:

- Busy waiting consumes CPU cycles.
- Not scalable for more than two processes without additional complexity.
- May not work correctly on modern systems with multiple CPUs due to memory reordering.